

Sprint 1 — Especificação do Projeto

Geladeira Zero: Combate ao Desperdício de Alimentos com IA

Disciplina:	Algoritmos e Programação de Computadores (APC)
Semestre:	2026/1
Professor:	Edison Ishikawa
Tema do projeto:	IA e meio ambiente
Linguagem:	Python 3.x
Nome equipe:	Cientistas da Computaria
Entrega da especificação:	27/05/2026
Apresentação final:	01/07/2026

Equipe (preencher com os nomes e matrículas)

#	Nome completo	Matrícula	Papel principal
1	Gabriel Vieira Franco	261031054	Líder / Integração / Interface
2	Pedro Vargas dos Santos e Silva	261023212	Módulo IA / Análise estatística
3	Jonas Morato Brandão	261042352	Inventário / Validades / Alertas
4	Felipe Vasques		Persistência / Dados / Impacto

1. Visão do Produto

1.1 Problema

O desperdício de alimentos é um dos maiores problemas ambientais e econômicos das famílias brasileiras. Estima-se que cerca de um terço de todo o alimento produzido no mundo seja desperdiçado, gerando emissões de gases de efeito estufa equivalentes a milhões de toneladas de

CO₂ por ano. No nível doméstico, o desperdício ocorre principalmente por três motivos: (1) perda de controle sobre o que existe na geladeira/despensa, (2) compra em excesso e (3) falta de criatividade para usar ingredientes próximos do vencimento.

1.2 Proposta

O **Geladeira Zero** é um software em Python que ajuda famílias e estudantes a reduzir o desperdício de alimentos por meio de três funções principais: (1) manter um inventário atualizado do que existe em casa, com data de compra e validade estimada, (2) alertar o usuário sobre itens próximos do vencimento e (3) usar IA Generativa para sugerir receitas que utilizem exatamente os ingredientes que estão prestes a estragar. O sistema também acompanha o impacto evitado (kg de alimento salvo e CO₂ não emitido) e calcula o dinheiro economizado.

2

1.3 Público-alvo

Estudantes universitários que moram sozinhos ou em repúblicas (planejam mal as compras) e famílias urbanas que querem economizar e reduzir desperdício. O produto deve funcionar sem exigir conhecimento prévio de nutrição ou culinária.

1.4 Diferencial e uso de IA Generativa

Aplicativos tradicionais de inventário doméstico apenas listam o que existe. O Geladeira Zero vai além: usa IA Generativa para criar receitas personalizadas a partir do que o usuário tem agora, considerando preferências (vegetariano, sem lactose, etc.) e tempo disponível. A IA atua como camada criativa, enquanto o controle de validades, alertas e cálculo de impacto seguem regras determinísticas auditáveis.

1.5 Resultado esperado

Ao final do projeto teremos: (a) software funcional em Python 3, (b) base com pelo menos 80 alimentos comuns e suas validades típicas, (c) integração com IA Generativa para receitas, (d) experimento controlado pesando o lixo orgânico de pelo menos 8 voluntários antes e depois do uso do sistema, e (e) repositório público no GitHub com código e documentação.

2. Requisitos

2.1 Requisitos funcionais

ID		Prioridade
RF01	O sistema deve apresentar um menu principal com as opções disponíveis.	Alta
RF02	O usuário deve poder cadastrar um alimento com nome, quantidade, data de compra e validade.	Alta
RF03	O sistema deve sugerir validade estimada com base em uma base de alimentos.	Alta
RF04	O sistema deve exibir o inventário ordenado por proximidade do vencimento.	Alta
RF05	O sistema deve gerar alertas para itens que vencerão nos próximos 3 dias.	Alta
RF06	O sistema deve sugerir receitas usando os itens mais próximos do vencimento (IA).	Alta
RF07	O sistema deve permitir marcar um item como consumido ou descartado.	Alta
RF08	O sistema deve calcular kg de alimento salvo e CO evitado.	Alta
RF09		Média
RF10	O sistema deve calcular dinheiro economizado (preço médio por kg).	Média
RF11	O sistema deve permitir registrar preferências alimentares (vegetariano, alergias).	Média
RF12	O sistema deve exportar o histórico para CSV.	Baixa

2.2 Requisitos não funcionais

ID	Requisito
RNF01	Implementado em Python 3.x, sem dependências de sistemas operacionais específicos.
RNF02	Todo o código deve ser comentado e documentado (docstrings nas funções).
RNF03	Toda entrada do usuário deve ser validada (datas, números, strings) — sem surpresas na execução.
RNF04	O sistema deve persistir dados entre execuções (arquivos JSON).
RNF05	A chamada à IA deve ter timeout e fallback para receitas pré-cadastradas.
RNF06	Tempo de resposta do menu (sem IA) deve ser inferior a 1 segundo.
RNF07	O código deve seguir PEP-8 e ser versionado no GitHub.

3. Desenho da Interface

Interface em modo texto (CLI), com menu numerado. A escolha por CLI permite focar nos algoritmos centrais (inventário, alertas, IA) sem desviar esforço para GUI. Versão com Tkinter (pacote não visto em APC, item bonificado) está planejada para o Sprint 4.

3.1 Tela inicial (menu principal)

```
+-----+
| GELADEIRA ZERO - Desperdicio Zero | +-----+
+-----+ | Ola, Joao! Itens na
geladeira: 14 | Vencem em 3 dias: 3 | | Alimento salvo este mes:
1.8 kg | Economia: R$ 24,30 | +-----+
+-----+ | | | [1] Adicionar item ao
inventario | | [2] Ver inventario (ordenado por validade) | | [3]
Ver alertas de vencimento | | [4] Sugerir receita com o que tenho
| | [5] Marcar item como consumido / descartado | | [6] Ver
impacto e economia | | [7] Configuracoes (preferencias
alimentares) | | [8] Exportar historico (CSV) | | [0] Sair | |
+-----+
Escolha uma opcao: _
```

3.2 Tela de alerta + sugestão de receita (RF05-RF06)

```
+-----+
| ALERTA DE VENCIMENTO | +-----+
+-----+ | | | 3 itens vencem nos proximos dias:
| | | | - Tomate (3 unid.) vence em 1 dia | | - Iogurte natural
vence em 2 dias | | - Manjericao fresco vence em 3 dias | | |
Deseja uma receita usando esses itens? [S/N]: S | | | +-----+
+-----+ | RECEITA
SUGERIDA (gerada por IA) | +-----+
+-----+ | | | Molho de tomate fresco com
manjericao e iogurte | | | | Ingredientes principais (do seu
inventario): | | - 3 tomates maduros | | - 4 folhas de manjericao
| | - 2 colheres de iogurte natural | | | | Voce ainda precisa
de: alho, sal, azeite | | Tempo: 20 minutos | Dificuldade: facil
| | | | Modo de preparo: | | 1. Pique os tomates em cubos
pequenos... | | [...] | | | +-----+
+-----+
```

[S] Salvar receita [O] Outra sugestao [0] Menu principal

3.3 Tela de impacto e economia (RF08-RF09)

```
+-----+
| MEU IMPACTO E ECONOMIA | +-----+
-----+ | | Perodo: ultimos 30 dias | | |
Itens consumidos no prazo: 18 | | Itens descartados: 4 | | Taxa
de aproveitamento: 82% | | | Alimento salvo: 1.8 kg | | CO2
evitado: 4.5 kg | | Agua economizada: 540 L (pegada hidrica) | |
Dinheiro economizado: R$ 24,30 | | | Categorias mais salvas: |
| Frutas/Legumes ##### 9 | | Laticinios #### 4 | | Carnes
### 3 | | | +-----+
-----+ [Enter] Voltar
```

4. Estrutura de Dados

Três entidades principais persistidas em arquivos JSON. JSON foi escolhido (em vez de SQLite) porque mapeia diretamente para dicionários e listas de Python, mantendo o código legível para alunos de APC.

4.1 Entidade Alimento (base de conhecimento)

```
alimento = {
    "id": int,
    "nome": str,          # ex.: "tomate"
    "sinonimos": list[str],
    "categoria": str,     # "fruta" | "legume" | "laticinio" | "carne" | ...
    "validade_dias": {
        "geladeira": int,
        "ambiente": int,
        "congelado": int
    },
    "preco_medio_kg": float,
    "co2_kg_por_kg": float,
    "agua_l_por_kg": float,
    "vegetariano": bool,
    "vegano": bool,
    "contem": list[str]   # alergenicos: "lactose", "gluten", ...
}
```

4.2 Entidade ItemInventario

```
item_inventario = {
    "id": int,
    "alimento_id": int,
    "quantidade": float,
    "unidade": str,       # "kg" | "g" | "L" | "unid"
    "data_compra": str,   # ISO 8601
    "data_validade": str, # ISO 8601
    "local": str,         # "geladeira" | "congelador" | "despensa"
    "status": str         # "ativo" | "consumido" | "descartado"
}
```

4.3 Entidade Usuário

```
usuario = {
    "nome": str,
    "data_cadastro": str,
    "preferencias": {
        "vegetariano": bool,
        "vegano": bool,
        "alergias": list[str],
        "tempo_max_min": int
    },
    "impacto_acumulado": {
        "alimento_salvo_kg": float,
        "co2_evitado_kg": float,
        "economia_reais": float
    }
}
```

4.4 Organização em arquivos

Arquivo	Conteúdo	Tipo
base_alimentos.json	Lista de alimentos (base de conhecimento, ~80 entradas).	leitura/escrita

inventario.json	Itens atualmente em casa.	leitura/escrita
historico.json	Itens consumidos ou descartados (para estatísticas).	leitura/escrita
usuario.json	Perfil único do usuário.	leitura/escrita
receitas_cache.json	Receitas geradas pela IA (cache de fallback).	leitura/escrita

5. Algoritmo Principal

Dois fluxos centrais: (a) atualização do inventário e geração de alertas, personalizada com IA. Abaixo o pseudocódigo de cada um.

(b) geração de receita

5.1 Fluxo principal (pseudocódigo)

```
INICIO
  carregar base_alimentos, inventario, historico, usuario
  verificar_alertas_de_vencimento()
  ENQUANTO usuario nao escolher sair:
    exibir_menu()
    opcao = ler_opcao()
    SE opcao == 1: adicionar_item()
    SENAO SE opcao == 2: ver_inventario()
    SENAO SE opcao == 3: ver_alertas()
    SENAO SE opcao == 4: sugerir_receita()
    SENAO SE opcao == 5: marcar_consumido_ou_descartado()
    SENAO SE opcao == 6: ver_impacto()
    SENAO SE opcao == 7: configuracoes()
    SENAO SE opcao == 8: exportar_csv()
    SENAO SE opcao == 0: sair = VERDADEIRO
    SENAO: exibir_mensagem("opcao invalida")

  FIM ENQUANTO
  salvar_dados()
FIM
```

5.2 Algoritmo de alerta de vencimento

```
FUNCAO verificar_alertas_de_vencimento():
  hoje = data_atual()
  alertas = lista_vazia()

  PARAcada item em inventario:
    SE item.status != "ativo": CONTINUE
    dias_restantes = item.data_validade - hoje
    SE dias_restantes <= 3:
      alertas.adicionar(item, dias_restantes)

  ORDENAR alertas POR dias_restantes (crescente)
  SE alertas nao vazio:
    exibir(alertas)
    perguntar_se_quer_receita()
FIM FUNCAO
```

5.3 Algoritmo de sugestão de receita (núcleo do produto)

```
FUNCAO sugerir_receita():
  itens_urgentes = filtrar(inventario, dias_validade <= 3)
  SE itens_urgentes esta vazio:
    itens_urgentes = filtrar(inventario, dias_validade <= 7)

  itens_filtrados = aplicar_preferencias(itens_urgentes, usuario.preferencias)

  prompt = montar_prompt(
    ingredientes      = itens_filtrados,
    restricoes        = usuario.preferencias.alergias,
    estilo_culinario  = usuario.preferencias.vegetariano,
    tempo_maximo      = usuario.preferencias.tempo_max_min
  )

  TENTAR:
    receita = consultar_ia(prompt, timeout=10s)
    salvar_em_cache(receita)
```

```

EMCASO DE ERRO/timeout:
    receita = buscar_no_cache(itens_filtrados)
SE nada no cache:
    receita = receita_generica(itens_filtrados)

exibir_receita(receita)
acao = perguntar([Salvar, Outra, Voltar])
SE acao == "Salvar":
    historico_receitas.adicionar(receita)
    salvar_dados()
FIM FUNCAO

```

5.4 Algoritmo de cálculo de impacto

```

FUNCAO calcular_impacto(periodo_dias):
    total_salvo_kg = 0.0
    total_co2_kg   = 0.0
    total_economia = 0.0

    PARA cada item em historico (no periodo):
        alimento = buscar_alimento(item.alimento_id)
        qtd_kg = converter_para_kg(item.quantidade, item.unidade)

        SE item.status == "consumido":
            total_salvo_kg += qtd_kg
            total_co2_kg   += qtd_kg * alimento.co2_kg_por_kg
            total_economia += qtd_kg * alimento.preco_medio_kg

    RETORNA (total_salvo_kg, total_co2_kg, total_economia)
FIM FUNCAO

```

5.5 Conteúdos de APC mobilizados pelo projeto

Até este ponto da disciplina, a turma já estudou: **variáveis e tipos primitivos**, **strings**, **estruturas condicionais** (if/elif/else), **iteração** (while e for), **listas**, **tuplas** e **funções**. A tabela abaixo mapeia cada conteúdo já visto ao seu uso concreto no projeto.

Conteúdo de APC	Como o projeto usa	Onde no código
Variáveis e tipos primitivos	Contadores de itens, totais de impacto (float), strings de descrição, booleanos de status.	Em todos os módulos.
Strings	Normalização de nomes de alimentos (lower(), strip(), remove acentos), comparação de sinônimos, montagem de prompts para a IA, formatação de telas e relatórios.	classificador, ia, interface
Condicionais (if/elif/else)	Roteamento do menu principal, lógica de alertas (dias_restantes <= 3), validação de entradas, escolha entre fonte local e IA, filtros de preferência alimentar.	main, alertas, ia
Iteração (while e for)	Loop principal do menu (while), percorrer o inventário para alertas (for), agregar valores no cálculo de impacto, leitura linha a linha do CSV exportado.	main, alertas, impacto
Listas	Inventário de itens, histórico de consultas e receitas, lista de alertas ordenada por urgência, ingredientes de uma receita.	inventario, alertas, historico
Tuplas	Retorno múltiplo de funções (kg_salvo, co2_kg, economia), pares (nome, quantidade) usados em ordenação estável de alertas, coordenadas (item, dias_restantes).	impacto, alertas
Funções	Cada operação em função própria com docstring: adicionar_item(), buscar_alimento(), verificar_alertas(), sugerir_receita(), calcular_impacto(), carregar_json(), salvar_json().	todos os módulos

Conteúdos que ainda serão estudados em APC e o projeto antecipa: dicionários (estrutura central das entidades JSON), manipulação de arquivos (JSON e CSV), tratamento de exceções (timeout da IA, JSON corrompido), e módulos (separação do código em arquivos). O projeto serve, portanto, como contexto motivador para fixar esses conteúdos futuros.

6. Cronograma

Sprints semanais cobrindo as 6 semanas entre 20/05 e 01/07. Cada sprint entrega um incremento funcional (metodologia iterativa) e produz um relatório de aprendizagem da equipe.

Sprint	Período	Entregáveis	Marco
S1	20/05 a 27/05	Especificação (este documento), repositório Git, base com 20 alimentos, menu funcional, CRUD básico do inventário.	27/05: upload no Moodle
S2	28/05 a 03/06	Cálculo de validade, alertas de vencimento, função de marcar consumido/descartado, base ampliada para 50 alimentos. Relatório de Aprendizagem #1.	03/06: Avaliação Intermediária (pitch)
S3	04/06 a 11/06	Integração da IA para receitas, cache de fallback, cálculo de impacto, slides e pôster do Fórum de Extensão.	11/06: 1ª Jornada de Extensão (pôster)
S4	12/06 a 18/06	Base completa (80+ alimentos), preferências do usuário, exportação CSV, interface gráfica Tkinter (bônus). Relatório de Aprendizagem #2.	Produto-piloto pronto para testes
S5	19/06 a 25/06	Execução do experimento com voluntários (mínimo 8 famílias, pesagem do lixo orgânico), coleta de dados e análise estatística com pandas.	Resultados experimentais coletados
S6	26/06 a 01/07	Correções finais, documentação, README, gravação do vídeo de 7 min, ensaio da apresentação. Relatório de Aprendizagem #3.	01/07: Apresentação final

7. Divisão de Tarefas na Equipe (4 integrantes)

A equipe é composta por 4 integrantes. Em vez de manter um papel exclusivo para “Experimento” (que tradicionalmente seria de uma 5ª pessoa), o experimento foi distribuído como responsabilidade compartilhada (ver seção 7.6), e a interface foi consolidada com o papel de Líder, já que quem coordena o sistema é quem melhor entende como ele se apresenta ao usuário. Cada papel cobre aproximadamente 25% do código.

7.1 Visão geral dos 4 papéis

#	Papel	Arquivos	Foco
Integrante 1	Líder / Integração / Interface	main.py, interface.py, README.md	Coordenação geral, telas, validação de entradas.
Integrante 2	Módulo IA / Análise estatística	ia.py, prompts.json, analise.ipynb	Engenharia de prompts, cache de fallback, análise dos dados do experimento.
Integrante 3	Inventário / Validades / Alertas	inventario.py, alertas.py, base_alimentos.json	CRUD do inventário, lógica de vencimentos, base de 80 alimentos.
Integrante 4	Persistência / Dados / Impacto	persistencia.py, impacto.py, historico.json	Leitura/escrita de JSON, exportação CSV, cálculo de CO ₂ e economia.

7.2 Integrante 1 — Líder / Integração / Interface

Por que esses três juntos: o líder precisa entender o sistema como um todo, e a interface é o ponto onde todos os módulos se encontram. Quem cuida da interface naturalmente vê o sistema como o usuário vê.

Arquivos:

- **main.py** — ponto de entrada, loop principal, roteamento do menu.
- **interface.py** — todas as telas, formatação, validação de entrada do usuário.
- **README.md** — documentação do projeto.
- Coordenação geral do Git (merges, resolução de conflitos).

Entregáveis por sprint:

Sprint	Entregável
S1	Menu principal funcional com as 9 opções navegáveis (algumas como stubs); função ler_opcao() validando entrada; README com instruções de instalação.
S2	Telas de listagem do inventário e de alertas; função limpar_tela() cross-platform; primeiro merge integrando trabalho dos 4 integrantes. Tela de receita sugerida pela IA; tela de impacto/economia; coordenar a montagem do pôster.
S3	Refinamento da UX (mensagens de erro, confirmações); tela de configurações de preferências; revisão final do README.
S4	Apoiar a coleta do experimento (UI para o experimento.py); merges finais.
S5	
S6	Gravação do vídeo de 7min; ensaio da apresentação; tag v1.0 no Git.

Conteúdos de APC que vai praticar mais: strings (formatação de telas), condicionais (roteamento do menu), iteração (loop principal, listagem), funções (decomposição).

7.3 Integrante 2 — Módulo IA / Análise estatística

Por que esses dois juntos: quem mexe com a API da IA já lida com requests, JSON e estruturas complexas. A análise estatística do experimento usa as mesmas ferramentas (pandas, manipulação de JSON), então é continuidade natural.

Arquivos:

- **ia.py** — chamada à API da IA Generativa, parser de resposta, cache.
- **prompts.json** — biblioteca de prompts versionados.
- **receitas_cache.json** — cache de fallback.
- **experimento/analise.ipynb** — análise estatística dos dados do experimento.

Entregáveis por sprint:

Sprint	Entregável
S1	Pesquisar 2-3 APIs candidatas (Gemini, OpenAI, Claude); criar conta gratuita; testar uma chamada manual via curl ou Postman; documentar limites de uso.
S2	Esqueleto do ia.py com função consultar_ia(prompt) mockada (retorna receita fixa enquanto a API não está integrada); estrutura inicial do prompts.json.
S3	Integração real com a API funcionando; engenharia de prompt para receitas; cache de fallback implementado; tratamento de timeout e erros HTTP.
S4	Refinamento dos prompts com base em testes; restringir a IA a ingredientes acessíveis; logging das chamadas para auditoria.
S5	Receber os dados do experimento e fazer a análise estatística (teste t pareado) no Jupyter Notebook usando pandas e scipy.
S6	Gerar gráficos finais para o pôster e slides (matplotlib); ajustes finais nos prompts; redação da seção de análise no relatório final.

Conteúdos de APC que vai praticar mais: strings (montagem de prompts), tratamento de erros (try/except no timeout), funções (cada chamada à IA isolada), dicionários (resposta JSON da API).

Atenção especial: esse papel é o mais “novo” — exige aprender requests e a API. Esse integrante precisa começar a estudar já no Sprint 1, mesmo sem código entregue imediatamente.

7.4 Integrante 3 — Inventário / Validades / Alertas

Por que esses três juntos: alertas são consequência direta do inventário (percorrer a lista e ver o que está vencendo). Quem cadastra os alimentos também conhece a base, então sabe quais alertas fazem sentido.

Arquivos:

- **inventario.py** — CRUD de itens (adicionar, listar, marcar consumido/descartado).
- **alertas.py** — lógica de vencimentos, ordenação por urgência.
- **data/base_alimentos.json** — a base de 80 alimentos.

Entregáveis por sprint:

Sprint	Entregável
S1	base_alimentos.json com 20 alimentos preenchidos manualmente (tomate, leite, arroz, frango, ovo, alface, banana, etc.); função adicionar_item() funcional; função listar_inventario().
S2	alertas.py completo: função verificar_alertas_de_vencimento() retornando lista ordenada; função marcar_consumido_ou_descartado(); base ampliada para 50 alimentos.

S3	Função buscar_alimento_por_nome() com tolerância (lower, sem acento, busca por sinônimo); sugestão automática de validade ao cadastrar item.
S4	Base ampliada para 80+ alimentos, com revisão cruzada por outro integrante; ajustes finais na lógica de vencimentos.
S5	Apoiar o experimento: cadastrar dados-piloto no inventário para os voluntários.
S6	Documentação interna da base (de onde veio cada dado: CEMPRE, EMBRAPA, etc.); ajustes finais.

Conteúdos de APC que vai praticar mais: listas (percorrer inventário), condicionais (lógica de vencimento), strings (normalização de nomes), iteração (loops sobre a base), tuplas (pares item + dias_restantes para ordenação).

Atenção especial: esse é o papel mais “trabalhoso em dados” — encher 80 alimentos com 8 campos cada dá ~640 valores. Recomenda-se fazer em planilha primeiro e depois converter para JSON.

7.5 Integrante 4 — Persistência / Dados / Impacto

Por que esses três juntos: quem cuida da leitura/escrita dos arquivos já tem visão de todos os dados que circulam pelo sistema. O cálculo de impacto (CO, economia) é só agregação dos dados₂ do histórico — usa exatamente o que esse integrante já lê e escreve.

Arquivos:

- **persistencia.py** — `carregar_json()`, `salvar_json()`, exportação CSV.
- **impacto.py** — cálculo de kg salvo, CO evitado, economia.
- **data/historico.json** — histórico de itens² consumidos/descartados.
- **data/usuario.json** — perfil e preferências do usuário.

Entregáveis por sprint:

Sprint	Entregável
S1	<code>persistencia.py</code> com <code>carregar_json()</code> e <code>salvar_json()</code> testados; criação automática da pasta <code>data/</code> se não existir; tratamento de arquivo inexistente.
S2	Gestão do <code>historico.json</code> : quando um item é marcado como consumido/descartado, mover para o histórico; gestão do <code>usuario.json</code> com preferências.
S3	<code>impacto.py</code> completo: funções <code>calcular_impacto_periodo()</code> , <code>agregar_por_categoria()</code> ; integração com a tela de impacto que o Líder está fazendo.
S4	Função <code>exportar_csv()</code> do histórico (RF11); validação de integridade dos JSONs (detectar corrompido).
S5	Apoiar o experimento: preparar o <code>pre_pos_teste.csv</code> e a planilha de pesagem usada pelos voluntários.
S6	Geração do CSV final com todos os dados do experimento, formatado para o <code>analise.ipynb</code> do Integrante 2.

Conteúdos de APC que vai praticar mais: funções (cada operação isolada), strings (parsing de datas, formatação CSV), listas e dicionários (estrutura dos JSONs), iteração (percorrer histórico para agregar).

Atenção especial: esse é o papel mais “transversal”. Os outros 3 integrantes vão depender desse módulo desde o S1. Por isso o entregável da S1 desse integrante é o mais crítico — sem ele, ninguém consegue salvar ou carregar nada.

7.6 Responsabilidades compartilhadas pelos 4 integrantes

Algumas atividades não pertencem a um único papel — são responsabilidade de toda a equipe. Cada integrante contribui individualmente:

- **Recrutar voluntários para o experimento:** cada um recruta 2-3 pessoas → 8-12 finalistas (com folga para abandono).
- **Code review:** pelo menos um revisor por pull request, citando o conceito de APC envolvido na revisão.
- **Relatório de Aprendizagem quinzenal:** cada integrante posta o seu, individual, nos sprints S2, S4 e S6.
- **Auto-avaliação por sprint:** cada um preenche a rubrica de 5 níveis da seção 9.3 ao final de cada sprint.
- **Pré-teste e pós-teste de APC:** cada um faz o seu, no S1 e no S6, para medir aprendizado individual.

- **Reuniões semanais:** 30 minutos toda semana (presencial ou online) para sincronizar progresso e dúvidas.
- **Apresentações públicas:** os 4 participam do pôster na Jornada (11/06) e da apresentação final (01/07).

7.7 Dependências entre integrantes

Para evitar bloqueios, esta tabela explicita quem precisa entregar antes para que outros possam começar. A ordem ideal de codificação na semana inicial é: **Integrante 4 → Integrantes 1, 2 e 3 em paralelo.**

Sprint	Quem entrega primeiro	Quem depende
S1	Integrante 4 (persistencia.py)	Todos os outros — sem leitura/escrita de JSON, ninguém testa nada.
S1	Integrante 3 (base_alimentos.json com 20 itens)	Integrante 1 (precisa de dados reais para testar o menu). Integrante 1 (precisa para fazer a tela de alertas).
S2	Integrante 3 (alertas.py)	Integrante 1 (precisa para fazer a tela de receita).
S3	Integrante 2 (ia.py integrada)	Integrante 1 (precisa para fazer a tela de impacto).
S3	Integrante 4 (impacto.py)	Integrante 2 (precisa para rodar a análise estatística).
S5	Integrante 4 (CSV consolidado)	

8. Metodologia Experimental (avaliação do produto)

O experimento responde à pergunta: “O Geladeira Zero reduz a quantidade de alimento desperdiçado por uma família/morador?” O desenho usa medidas reais (peso do lixo orgânico) antes e depois do uso do sistema, com metodologia científica.

8.1 Desenho do experimento

- **Participantes:** mínimo 8 voluntários (estudantes ou famílias) por 4 semanas. Recrutamento compartilhado pelos 4 integrantes (2-3 cada).
- **Material:** balança de cozinha emprestada/comprada pela equipe, planilha de registro.
- **Fase 1 (linha de base):** nas 2 primeiras semanas, voluntário pesa todo o lixo orgânico descartado por estragar (sem usar o sistema).
- **Fase 2 (tratamento):** nas 2 semanas seguintes, voluntário usa o Geladeira Zero diariamente e continua pesando o lixo orgânico.
- **Variável dependente:** kg de alimento desperdiçado por semana por participante.
- **Análise estatística:** teste t de Student pareado (mesmo participante em duas condições), com nível de significância $\alpha = 0,05$. Conduzida pelo Integrante 2.
- **Hipótese nula (H_0):** não há diferença no desperdício entre as fases.
- **Hipótese alternativa (H_1):** o desperdício é menor na Fase 2 (com Geladeira Zero).
- **Generalização:** os resultados serão estratificados por perfil (estudante solo vs. família), e a discussão final identifica condições sob as quais o efeito se mantém.

8.2 Métricas adicionais coletadas

- Número de receitas sugeridas que o usuário efetivamente fez.
- Tempo médio gasto para cadastrar um item no inventário (UX).
- Avaliação subjetiva (escala Likert 1–5) sobre utilidade do sistema.
- Economia em reais relatada pelos participantes ao final.

9. Avaliação do Aprendizado da Equipe em APC

Conforme exige o enunciado (slide 8 do briefing), além de avaliar o produto, a equipe deve aplicar uma metodologia para medir se a forma como o projeto foi conduzido “serviu para aprender a programar melhor”, ou seja, se fixou o conteúdo de APC.

9.1 Pergunta de avaliação do aprendizado

“O uso da IA Generativa durante o desenvolvimento do Geladeira Zero ajudou ou atrapalhou a fixação dos conteúdos de APC pela equipe?” Como o slide 8 sugere, **o resultado da P2 pode ser uma medida objetiva:** cada integrante comparará sua nota da P2 (após o projeto) com a da P1 (antes do projeto).

9.2 Instrumentos de coleta

Instrumento	Descrição
Pré-teste individual	Aplicado na primeira reunião (S1). 10 questões sobre iteração, condicionais, strings, listas, tuplas e funções. Repetido como pós-teste na S6.

Relatórios quinzenais de aprendizagem	Cada integrante posta no Moodle 3 relatórios (S2, S4, S6) refletindo sobre: (a) o que aprendeu, (b) o que a IA fez por ele vs. o que ele fez sozinho, (c) o que ainda não entende.
Auto-avaliação por sprint	Ao final de cada sprint, cada integrante preenche a rubrica da seção 9.3 (1-5) para os conteúdos mobilizados.
Peer review	Em cada pull request, o revisor cita o conceito de APC envolvido (ex.: “sua condição poderia usar elif para evitar avaliação redundante”).
Comparação P1 vs. P2	Variação individual de nota da P1 (antes) para a P2 (após) como medida objetiva agregada.
Entrevista final	Conversa de 15 min com cada integrante na S6, gravada e transcrita, para captura qualitativa.

9.3 Rubrica de auto-avaliação por sprint (escala 1–5)

Nível	Descrição
1 — Não domino	Não consigo escrever o código sem ajuda externa (IA ou colega).
2 — Domino com IA	Consigo escrever com a IA gerando o código; eu reviso e corrijo.
3 — Domino com referência	Consigo escrever consultando material da disciplina; pouca ajuda da IA.
4 — Domino sozinho	Consigo escrever sem consultas; reconheço e corrijo erros sozinho.
5 — Domino e ensino	Consigo explicar o conceito a um colega e identificar abordagens alternativas.

9.4 Análise dos dados de aprendizagem

Ao final do projeto serão computados, para cada integrante: (a) variação pré→pós-teste, (b) progressão da auto-avaliação ao longo dos sprints, (c) variação P1→P2. A comparação P1 vs. P2 é analisada com teste t pareado, mesmo procedimento do experimento do produto.

10. Uso de Pacotes Não Vistos em APC (item bonificado)

O enunciado bonifica o uso de pacotes e elementos não estudados em APC. A equipe planeja usar os seguintes pacotes, todos com integração documentada:

Pacote	Uso no projeto	Sprint	Responsável
json (biblioteca padrão)	Persistência das entidades em arquivos JSON.	S1	Integrante 4
csv (biblioteca padrão)	Exportação do histórico para análise em planilha.	S4	Integrante 4
datetime (biblioteca padrão)	Cálculo de dias até vencimento, formatação de datas.	S2	Integrante 3
requests	Chamada HTTP à API da IA Generativa, timeout e tratamento.	S3	Integrante 2
tkinter	Interface gráfica opcional (GUI desktop).	S4	Integrante 1
pandas + matplotlib	Análise estatística do experimento e gráficos.	S5	Integrante 2

11. Repositório no GitHub

Repositório público no GitHub, sob o nome **geladeira-zero-apc-2026-1**,
pastas: licença MIT. Estrutura de

```
geladeira-zero-apc-2026-1/
### README.md           # visao geral, instalacao, uso
### LICENSE             # MIT
### requirements.txt    # dependencias
### .gitignore
### docs/
#   ### sprint1_especificacao.pdf
#   ### poster.pdf
#   ### slides_final.pdf
#   ### relatorios_aprendizagem/
#       ### relatorio_1.pdf
#       ### relatorio_2.pdf
#       ### relatorio_3.pdf
### src/
#   ### main.py          # Integrante 1
#   ### interface.py     # Integrante 1
#   ### gui.py           # Integrante 1 (Tkinter, bonus)
#   ### ia.py            # Integrante 2
#   ### inventario.py    # Integrante 3
#   ### alertas.py       # Integrante 3
#   ### persistencia.py  # Integrante 4
#   ### impacto.py       # Integrante 4
### data/
#   ### base_alimentos.json # Integrante 3
#   ### prompts.json      # Integrante 2
#   ### inventario.json   # gitignored
#   ### historico.json    # gitignored
#   ### usuario.json      # gitignored
### tests/
#   ### test_inventario.py
#   ### test_alertas.py
### experimento/
#   ### planilha_pesagem.csv
#   ### resultados_brutos.csv
#   ### pre_pos_teste.csv
```

11.1 Fluxo de trabalho no Git

- Branch principal: **main** (apenas código revisado e funcional).
- Branch de desenvolvimento por integrante: **feature/<nome>-<modulo>**.
- Pull request obrigatório para merge em main, com pelo menos 1 revisão (peer review).
- Commits em português, no formato “<modulo>: descrição curta”.
- Tag de versão ao final de cada sprint (v0.1, v0.2, ..., v1.0).

12. Início da Implementação (Sprint 1)

Ao final do Sprint 1 (27/05), os seguintes artefatos estarão prontos no repositório, com responsável claro:

Artefato	Descrição	Responsável
main.py	esqueleto do menu principal navegável (algumas opções como stubs) função <code>exibir_menu()</code> + validação de entrada do usuário	Integrante 1
interface.py	funções <code>carregar_json()</code> e <code>salvar_json()</code> testadas	Integrante 1
persistencia.py	funções <code>adicionar_item()</code> e <code>listar_inventario()</code>	Integrante 4
inventario.py	20 alimentos iniciais cadastrados	Integrante 3
data/base_alimentos.json	função <code>consultar_ia()</code> mockada retornando receita fixa	Integrante 3
ia.py (stub)	instruções de instalação e screenshot do menu rodando	Integrante 2
README.md		Integrante 1

13. Socialização e Integração com o CIC

Como objetivo explícito do projeto (slide 9 do briefing), a equipe se compromete a:

- Apresentar o produto na **1ª Jornada de Extensão do IE** em 11/06, expondo o pôster a alunos de outros semestres e professores do CIC.
- Recrutar voluntários para o experimento entre colegas de outras turmas de APC, criando ponto de contato natural com a comunidade do CIC.
- Divulgar o repositório público em grupos da turma e do centro acadêmico, convidando contribuições e issues.
- Após a apresentação final, manter o repositório aberto e ofertar o software como base para projetos de outras disciplinas.

14. Riscos e Mitigações

Risco	Impacto	Mitigação
Indisponibilidade da API de IA Generativa	Alto	Cache local de receitas geradas anteriormente; fallback para receitas pré-cadastradas.
Validade estimada incorreta para alguns alimentos	Médio	Permitir que o usuário corrija manualmente a data; revisão da base por pelo menos 2 integrantes.

Voluntários abandonarem o experimento (4 semanas)	Alto	Recrutar 10-12 participantes para garantir 8 finalistas; mensagens semanais de incentivo.
Equipe sem balança de cozinha disponível	Médio	Comprar balança barata (~R\$ 40) ou usar a do refeitório universitário.
IA sugerir receita inviável (ingredientes raros)	Médio	Refinar prompt para restringir a ingredientes acessíveis; validação pós-geração.
IA fazer o código por nós e prejudicar o aprendizado	Alto	Regra interna: ninguém faz commit de código gerado pela IA sem entender linha a linha.
Conflitos de merge no Git entre integrantes	Baixo	Pull requests pequenos e frequentes; reuniões semanais de sincronização.
Sobrecarga do Integrante 4 (todos dependem dele no S1)	Médio	Priorizar persistencia.py como primeira tarefa absoluta da semana; pareamento com outro integrante para revisão rápida.

Documento gerado para o Sprint 1 — Especificação do Projeto. Equipe de 4 integrantes. Próximo marco: Avaliação Intermediária em 03/06/2026, com entrega do primeiro Relatório de Aprendizagem.